

Constraint-Based Optimization for Weekly Training Program Design

A Greedy Heuristic Approach to Multi-Objective Scheduling

Author: Aaron Tobias

Abstract

Designing effective weekly training programs requires balancing multiple competing factors, including exercise selection, fatigue management, time constraints, recovery requirements, and user-specific goals. In practice, these decisions are often made heuristically rather than through formal optimization methods. This project proposes **AuraFit**, a system that formulates workout planning as a constrained scheduling problem solved via **Evolutionary Computation**. AuraFit utilizes a **Genetic Algorithm (GA)** to navigate the high-dimensional search space, ensuring global optimization where traditional greedy methods fail. The system integrates hard constraints, such as session duration and fatigue limits, with multi-objective fitness criteria, including movement taxonomy coverage and workload variance minimization. Results demonstrate that this evolutionary approach produces superior, constraint-compliant schedules by eliminating "look-ahead blindness" common in sequential scheduling.

Problem Definition and Motivation

Designing effective training programs is a complex task that requires balancing multiple competing factors, including movement coverage, fatigue management, time constraints, recovery requirements, and individual preferences. In practice, these programs are often constructed heuristically by coaches or practitioners based on experience rather than formal optimization. Drawing from experience in applied training and movement-based environments, it becomes evident that programming decisions are highly interdependent. Each exercise selection influences subsequent decisions due to accumulated fatigue, movement redundancy, and recovery limitations. This creates a structured decision-making problem that closely resembles the Resource-Constrained Project Scheduling Problem (RCPSP) studied in computer science (Pinedo, 2016). Despite this, most real-world training programs are not generated through formal algorithmic frameworks. Instead, they rely on informal heuristics that may not consistently optimize for key outcomes such as balanced workload distribution or efficient use of limited session time. This gap motivates the formulation of training program design as a computational problem.

In this project, weekly workout planning is modeled as a constrained optimization problem in which exercises must be assigned to training sessions under strict feasibility requirements. These include session duration limits, fatigue capacity, equipment availability, and inter-session recovery constraints. Fatigue is modeled as a bounded resource, reflecting well-established concepts of training load (Impellizzeri et al., 2005;

Zatsiorsky & Kraemer, 2006). Additionally, the system must satisfy higher-level objectives such as movement category coverage and alignment with user-specific goals, consistent with research on training frequency and adaptation (Schoenfeld et al., 2016). The system must operate efficiently under bounded computational complexity, producing schedules in near real-time while maintaining correctness under strict feasibility constraints.

Related Work and Industry Context

The problem of scheduling under constraints has been extensively studied in computer science, particularly in areas such as job scheduling, resource allocation, and combinatorial optimization (Cormen et al., 2009; Garey & Johnson, 1979). Classical approaches include greedy algorithms, dynamic programming, and approximation algorithms for NP-hard problems. Greedy algorithms are commonly used when optimal solutions are computationally expensive to obtain, providing efficient approximations through locally optimal decisions (Kleinberg & Tardos, 2006). Constraint satisfaction frameworks further extend this approach by separating feasibility constraints from optimization objectives (Dechter, 2006).

The problem of scheduling under constraints has been extensively studied in areas such as combinatorial optimization and job scheduling. Classical approaches include greedy algorithms and dynamic programming. Greedy algorithms provide efficient approximations but often fall into local optima.

In industry, similar principles are applied in large-scale systems such as recommendation engines, infrastructure scheduling, and logistics planning (Beyer et al., 2016; Gomez-Urbe & Hunt, 2015). These systems often rely on heuristic methods due to scalability constraints. However, existing approaches typically assume abstract task structures and do not incorporate domain-specific considerations such as fatigue accumulation or recovery constraints. This project addresses that gap by embedding domain-informed constraints into a scheduling framework.

Proposed Solution

The proposed system constructs weekly training plans using a constraint-based greedy scheduling algorithm, enhanced by a local search optimization. The solution separates decision-making into four layers:

1. A feasibility layer enforcing hard constraints
2. A scoring layer evaluating candidate exercises
3. A construction layer (Greedy heuristic) assembling the initial schedule
4. A refinement layer (Local Search) exploring schedule mutations to escape local minima and balance fatigue (Russell & Norvig, 2020)

This layered approach ensures that all generated plans satisfy strict constraints while prioritizing high-quality selections through heuristic scoring and subsequent refinement.

Algorithms and Data Structures

The core algorithm follows an evolutionary pipeline. The time complexity of this search is $O(G \times P \times E)$ where G represents total generations, P represents population size, and E represents the cost of the fitness evaluation. This results in linear scaling relative to population growth, ensuring efficiency for moderate-to-large exercise datasets.

The GA iterates over generations, using Elitism to preserve the top 5% of individuals while crossover swaps "training blocks" to preserve temporal recovery patterns. Memory usage remains efficient, requiring only the storage of current and next-generation populations alongside exercise metadata.

Evolutionary Design Rationale

A Genetic Algorithm was chosen to address the combinatorial complexity and myopic nature of greedy heuristics. While greedy systems select locally optimal choices, they cannot guarantee global optimality and often front-load the training week. By evaluating the entire 7-day microcycle simultaneously, AuraFit bridges the gap between local and global optimality, flattening the fatigue curve and balancing the workload

Fitness Scoring Function

The fitness judge evaluates candidates based on a weighted sum of factors:

- Exercise Priority & Goal Alignment
 - Taxonomy Coverage (9 core movement categories)
 - Session Density Reward (Preventing "Lazy AI" strategies)
 - Fatigue & Overlap Penalties
-

Constraint Modeling and Domain Representation

A key aspect of this project is the translation of domain-specific training principles into formal computational constraints. Exercises are modeled as structured entities with attributes influencing feasibility and desirability. Fatigue is treated as a bounded resource within each session, ensuring workload remains within manageable limits

(Impellizzeri et al., 2005). Recovery constraints enforce a 48-to-72 hour spacing between similar movement categories to allow for central nervous system and muscle protein synthesis repair (Haff & Triplett, 2015), preventing excessive repetition and reflecting real-world training practices (Zatsiorsky & Kraemer, 2006). Category coverage ensures structural completeness across the week, aligning with research emphasizing the importance of balanced training frequency (Schoenfeld et al., 2016).

System Architecture

The system is composed of modular components:

Input (JSON/SQLite) -> Models -> GA Engine (Selection/Breeding) -> Evaluator -> Final Plan

Each module handles a specific responsibility, improving clarity and extensibility.

Experimental Plan

The experimental evaluation compares the greedy scheduler and the refined local search output to a random feasible baseline. Experiments will be conducted on a standard computing environment across 30-trial batches to ensure statistical consistency and evaluate variance. The target structure focuses on multi-day (e.g., 5-day) training splits. Metrics include:

- • coverage score
- • priority score
- • time utilization
- • fatigue balance (variance minimization)
- • frequency goals (targeting 5 days/week)
- • constraint violations
- • total score

Visualization will include score comparisons and distribution analysis.

Limitations and Future Work

The greedy algorithm is inherently limited by its local decision-making strategy and may produce suboptimal global solutions if unrefined. The fatigue model is simplified as a flat resource metric and does not capture all complex physiological factors. Additionally, the scoring function relies on manually tuned weights, which may not generalize across all use cases. Future iterations of this project will transition from 'Flat Fatigue' optimization

to Undulating Periodization models to map against real-world macrocycle planning (Fleck & Kraemer, 2014).

Conclusion

By treating training as a resource-constrained optimization problem, AuraFit provides a robust solution to the complexities of human performance scheduling. The transition from greedy heuristics to evolutionary computation enables the model to satisfy multi-objective goals that are physically impossible to manage manually.

Implementation Roadmap

Week 1: core structures, loader, constraints

Week 2: scoring, evaluation, baseline

Week 3: refinement, experiments, analysis

Architecture Diagram

